inherit from `StatusCodeResult` and set the appropriate status code for their type (`400`, `200`, and `404`, respectively). Using these wrapper classes makes the intention of your code clearer than relying on other developers to understand the significance of the various status code numbers.

Once you've returned an `ActionResult` (or other object) from your controller, it's serialized to an appropriate response. This works in several ways, depending on

- The formatters that your app supports
- The data you return from your method
- The data formats the requesting client can handle

You'll learn more about formatters and serializing data in section 20.6, but before we go any further, it's worth zooming out a little and exploring the parallels between traditional server-side rendered applications and web API endpoints. The two are similar, so it's important to establish the patterns that they share and where they differ.

20.2 Applying the MVC design pattern to a web API

In ASP.NET Core, the same underlying framework is used in conjunction with web API controllers, Razor Pages, and MVC controllers with views. You've already seen this yourself; the web API `FruitController` you created in section 20.2 looks similar to the MVC controllers you saw in chapter 19.